

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

ELECTRONICS AND COMMUNICATION ENGINEERING DEPARTMENT

PROJECT MODULE – 3

AI/ML-BASED PREDICTIVE MAINTENANCE AND VEHICLE HEALTH MONITORING SYSTEM

Machine-learning driven engine-condition prediction, vehicle health scoring, maintenance alert generation and Explainable AI

PROJECT FOCUS

The project takes engine sensor observations, prepares them for machine learning, compares Logistic Regression, Random Forest and XGBoost, and translates the resulting prediction into failure probability, vehicle health score and maintenance status.

Project detail	Information
Student	Sagnik Bhattacharyya
Department	Electronics and Communication Engineering
Institution	Institute of Engineering and Management, Kolkata
Project area	Automotive AI / Machine Learning / Predictive Maintenance
Development environment	Google Colab / Python
Primary dataset	Predictive Maintenance Engine Sensor Dataset
Repository	github.com/SAGNIK890/Predictive-Maintenance-and-Vehicle-Health-Monitoring-System

The report follows the same visual and organizational language as the supplied Module 6 project-report reference: numbered sections, compact technical prose, evidence tables, diagrams, figures and engineering callouts. The content has been rewritten around the predictive-maintenance implementation documented in the project repository.

Implementation status	Status
Dataset acquisition	Implemented
Data preparation and exploratory analysis	Implemented
Multiple ML model training	Implemented
Failure probability estimation	Implemented
Vehicle health score	Implemented
Maintenance alert logic	Implemented
Explainable AI	Implemented

CONTENTS

S.NO	TOPICS	PAGE NO.
1	Executive Summary	3
2	Problem Statement, Objectives and Engineering Requirements	4
3	Overall Python / ML Architecture	5
4	Conceptual Predictive-Maintenance Pipeline	6
5	Dataset and Sensor Inputs	7
6	Data Preparation and Exploratory Analysis	8
7	Machine Learning Models and Training	9
8	Model Evaluation and Comparative Results	10
9	Feature Importance and Explainable AI	11
10	SHAP-Based Local Explanation	12
11	Failure Probability and Vehicle Health Score	13
12	Maintenance Status and Alert Generation	14
13	Testing and Observed Results	15
14	Implementation Process and Design Decisions	16
15	Project Outputs and Reproducibility	17
16	Validation, Limitations and Engineering Assumptions	18
17	Future Scope and Advanced Development Roadmap	19
18	Conclusion	20

REPORT ORGANIZATION

REPORT ORGANIZATION

The report follows the supplied reference report's sequence: executive summary, engineering requirements, architecture, implementation model, dataset and development details, evaluation evidence, testing, validation, future scope and conclusion. The same compact tables and engineering callouts are retained, while the technical subject has been changed from Android Automotive media playback to predictive vehicle health monitoring.

EVIDENCE CATEGORY

Evidence category	Included in this report
Project definition	Predictive-maintenance objective and engineering requirements
Dataset	Public engine sensor dataset and six implemented sensor features
Development	Google Colab notebook, Python pipeline and model-training workflow
Machine learning	Logistic Regression, Random Forest and XGBoost comparison
Explainability	Random Forest feature importance and SHAP contribution analysis
Outputs	Failure probability, health score, status, alerts and exported CSV/model files
Repository	Public GitHub repository containing the implementation artifacts

The report deliberately distinguishes what has actually been implemented from future extensions such as LSTM, GRU, Transformer models and live vehicle telemetry. This keeps the engineering record aligned with the repository rather than presenting planned functionality as completed functionality.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

1. Executive Summary

The project presents a practical machine-learning workflow for predictive maintenance and vehicle health monitoring. Instead of waiting for a vehicle component to fail or relying only on fixed service intervals, the system uses engine sensor observations to learn patterns associated with the recorded engine condition. The implementation is deliberately compact: a public engine-sensor dataset is loaded into a Google Colab notebook, cleaned, explored and divided into training and testing sets. Three classification approaches are then trained and compared before the Random Forest model is used for the project's vehicle-health demonstration.

The implemented sensor inputs are Engine rpm, Lub oil pressure, Fuel pressure, Coolant pressure, lub oil temp and Coolant temp. The target is Engine Condition. The repository contains the notebook, dataset reference, model-comparison results, feature-importance results, SHAP explanation, vehicle-health predictions and supporting project files.

■cite■turn1view0■turn3view0■turn3view1■turn3view2■

The model comparison gives a useful picture of the current prototype. XGBoost records 66.68% accuracy and 69.75% ROC-AUC, Logistic Regression records 66.11% accuracy and 69.18% ROC-AUC, while Random Forest records 65.11% accuracy and 68.12% ROC-AUC. These values are repository outputs rather than claims about production-vehicle performance.

■cite■turn3view0■

Project outcome	Observed / implemented status
Dataset loading	Implemented through direct CSV URL
Sensor preprocessing	Implemented with median imputation; scaling used for Logistic Regression
Classification models	Logistic Regression, Random Forest and XGBoost
Evaluation	Accuracy, Precision, Recall, F1 and ROC-AUC
Vehicle-health demonstration	Random Forest probability converted to health score
Maintenance state	HEALTHY / WARNING / CRITICAL demonstration thresholds
XAI	Random Forest importance and SHAP local contribution
Export	CSV files, serialized Random Forest and ZIP output workflow

ENGINEERING INTERPRETATION

The strongest part of the prototype is the traceable path from raw sensor values to a measurable prediction and then to an interpretable health output. The current accuracy level also makes the limitation visible: the model should be treated as an academic demonstration until it is calibrated and validated against representative fleet data.

The project therefore demonstrates the complete machine-learning lifecycle rather than only a trained classifier: data acquisition, preparation, modelling, evaluation, explainability and user-facing interpretation.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

2. Problem Statement, Objectives and Engineering Requirements

A vehicle continuously produces measurements that describe how its engine is operating. RPM, pressure and temperature signals can change as the engine moves between operating conditions. A useful predictive-maintenance system should turn these numerical observations into information that can be acted upon without requiring the user to inspect every model output manually.

The engineering problem addressed here is therefore not simply classification. The project needs a complete chain: sensor observations must reach a trained model consistently; the model must provide a probability-like output; that output must be converted into a health representation; and the resulting state must be easy to inspect and reproduce.

The repository describes the objective as predicting vehicle/component failure before it occurs and converting the machine-learning output into failure probability, vehicle health score, status, maintenance alert and contributing sensor factors. [■cite■turn0view0■](#)

Implemented objectives include dataset inspection, exploratory analysis, missing-value handling, train/test splitting, multiple model training, model comparison, failure-probability calculation, health-score calculation, alert generation, feature-importance analysis, SHAP analysis and export of the trained model and result files. [■cite■turn1view0■](#)

Requirement	Implemented behaviour	Evidence
Sensor-based input	Six engine-sensor variables are used	Notebook / dataset
Data preparation	Median imputation and model-specific scaling	Colab pipeline
Classification	Binary Engine Condition target	Training workflow
Model comparison	Three classification models evaluated	model_comparison_results.csv
Probability output	Class probability generated by trained models	Prediction function
Health representation	Health = $100 \times (1 - \text{failure probability})$	Notebook logic
Alerting	Threshold-based HEALTHY/WARNING/CRITICAL states	Monitoring function
Explainability	Global feature importance + local SHAP	XAI output files
Reproducibility	Fixed random state and direct dataset URL	Notebook

ENGINEERING PRINCIPLE

The prototype keeps measured implementation separate from conceptual automotive integration. The notebook implements the data-to-prediction path; it does not claim to be a certified ECU diagnostic system, a live CAN gateway or a production maintenance decision engine.

This separation is important because it makes the report technically honest while still showing how the same software architecture could later be connected to richer vehicle telemetry.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

3. Overall Python / ML Architecture

The project is organised around a conventional machine-learning workflow. The dataset is acquired from a public repository and read directly into Pandas. After inspection, the sensor columns become the feature matrix and Engine Condition becomes the target. A stratified 80:20 split is used so that the test set remains representative of the class proportions present in the dataset.

The model layer contains three different approaches. Logistic Regression provides a simple baseline and uses median imputation followed by standard scaling. Random Forest provides an ensemble of decision trees and feature importance. XGBoost provides a gradient-boosted tree approach for comparison. The resulting models expose probability estimates, which are later used by the monitoring layer.

Layer	Input	Processing	Output
Dataset	Public engine CSV	Pandas loading and inspection	DataFrame
Preparation	Sensor columns + target	Imputation / scaling where required	Training-ready data
Model	Training features	LR / RF / XGBoost	Class + probability
Evaluation	Predictions + labels	Accuracy / Precision / Recall / F1 / ROC-AUC	Comparison table
Monitoring	Failure probability	Health-score and threshold logic	Health + status
XAI	Trained Random Forest / sample	Importance + SHAP	Contributing factors
Export	Results and model	CSV / Joblib / ZIP	Reusable artifacts

DATA SOURCE	→	PREPROCESSING	→	ML MODELS
Engine sensor observations	→	Imputation + split	→	Logistic / RF / XGBoost
Engine Condition	→	Train / test	→	Class probability
↓		↓		↓
MODEL OUTPUT	→	HEALTH LAYER	→	EXPLANATION
Failure probability	→	Health score + status	→	Importance + SHAP

Figure 3.1. Development-to-monitoring organisation of the predictive-maintenance system.

The architecture intentionally keeps the core prediction mechanism independent from the final presentation. This means the same trained model could later feed a dashboard, Android application, fleet-monitoring service or edge device without changing the basic learning formulation.

IMPLEMENTATION BOUNDARY

The repository implements the software pipeline and demonstration outputs. Physical sensor acquisition, CAN/OBD-II integration and production-grade automotive control remain outside the current implementation scope.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

4. Conceptual Predictive-Maintenance Pipeline

The central idea is simple enough to follow from one sensor reading to one maintenance message. A vehicle observation enters the system as a row of numerical values. The preprocessing layer makes that row suitable for the trained model. The classifier then estimates the likelihood of the maintenance/fault class. Instead of exposing only a 0 or 1 label, the monitoring layer keeps the probability so that a continuous health value can be displayed.

For example, if a model produces a maintenance/fault probability of 0.28, the project's health-score formula produces 72%. If the probability rises to 0.70, the demonstration status moves into the CRITICAL band. These thresholds are project-level rules and are not presented as automotive safety standards.

Layer	Responsibility	Example
Sensor input	Provide measurable engine state	RPM, oil pressure, coolant temperature
Data preparation	Make observations consistent for training	Median imputation
Classifier	Learn relation between sensors and condition	Random Forest
Probability	Represent model confidence for the target class	0.28
Health score	Give a simple 0–100 representation	72%
Status	Translate probability into a readable state	HEALTHY
Alert	Tell the user whether inspection is indicated	NO IMMEDIATE ACTION
XAI	Show influential variables	Engine rpm / pressure / temperature

USER / OPERATOR	OBSERVATION	MODEL	ACTION
Reviews vehicle state	Six sensor values	Predicts condition	Health and alert
Changes operating condition	New observation	Recomputes probability	Updated state

Figure 4.1. Conceptual command-and-observation flow adapted to predictive maintenance.

This arrangement is useful because each stage can be inspected separately. A poor result can be investigated at the data level, the model level or the interpretation layer rather than treating the whole system as one opaque block.

TRACEABILITY

A single prediction can be traced back to the sensor row used as input, the model probability, the health-score transformation, the status threshold and, where required, the XAI explanation.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

5. Dataset and Sensor Inputs

The implementation uses the Predictive Maintenance Engine Sensor Dataset available through Hugging Face. The repository states that the selected dataset contains approximately 19,535 engine records and that the notebook downloads the CSV directly. [cite turn1view0](#)

The dataset is useful for this project because it already places the sensor measurements and the engine-condition target in a tabular form that can be consumed by conventional classification algorithms. It also avoids the need to invent sensor values for a demonstration.

Feature	Role in project	Interpretation
Engine rpm	Input	Engine rotational speed
Lub oil pressure	Input	Lubrication-system pressure signal
Fuel pressure	Input	Fuel-system pressure signal
Coolant pressure	Input	Cooling-system pressure signal
lub oil temp	Input	Lubricating-oil temperature
Coolant temp	Input	Coolant temperature
Engine Condition	Target	Binary engine-condition label

The project brief contains a broader list of possible automotive signals, including vibration, battery voltage, vehicle speed, fuel consumption and operating hours. Those fields are not present in the selected dataset and therefore are not fabricated in this implementation. This choice keeps the experiment reproducible and makes the report clear about what the model actually sees. [cite turn1view0](#)

DATASET SCOPE

The current model is a six-feature engine-sensor classifier. Adding vibration, battery, speed, consumption and operating-hours signals is a future data-integration task, not an already completed part of this repository.

The direct dataset URL used by the notebook is:

https://huggingface.co/datasets/mukherjee78/predictive-maintenance-engine-data/resolve/main/raw_data.csv

Dataset characteristic	Current implementation
Approximate records	19,535
Feature type	Numerical sensor measurements
Learning task	Binary classification
Target	Engine Condition
Acquisition method	Direct CSV URL in Colab
Manual raw-data upload required	No

Figure 5.1. Dataset organisation used by the notebook: six sensor inputs feed a binary engine-condition target.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

6. Data Preparation and Exploratory Analysis

Before a model can be trusted enough for an experiment, the data itself needs to be understood. The notebook therefore begins by inspecting the shape of the dataset, displaying sample rows, listing the available columns and checking missing values. Descriptive statistics are then used to see the numerical scale and spread of the sensor variables.

The target distribution is also inspected before training. This matters because a classifier can appear strong on accuracy while performing poorly on the class that matters most. The notebook uses a stratified train/test split to reduce this risk during the evaluation stage.

Preparation step	What is done	Reason
Load	Read CSV into Pandas	Create structured working data
Inspect	Shape, columns and first rows	Confirm dataset structure
Missing values	Check every feature	Detect incomplete observations
Statistics	Describe numerical variables	Understand scale and spread
Distribution	Plot feature histograms	See approximate feature behaviour
Correlation	Compute numeric correlation matrix	Inspect relationships between variables
Target	Count Engine Condition classes	Understand classification balance
Split	80/20 stratified split	Create reproducible evaluation set

For the model pipeline, median imputation is used where missing values are encountered. Logistic Regression additionally receives standard scaling because the model benefits from features being represented on comparable numerical scales. Random Forest and XGBoost are tree-based and do not require the same scaling step.

The use of Scikit-learn pipelines also reduces the chance of applying training-time preprocessing differently during inference. That consistency becomes especially important once a trained model is saved and reused.

Exploratory output	Purpose
Feature histograms	Understand the distribution of each sensor
Correlation matrix	Inspect pairwise numerical relationships
Target counts	Understand class representation
Descriptive statistics	Provide numerical context for the features
DATA-QUALITY PRINCIPLE	
The project does not create artificial vibration, battery or vehicle-speed columns merely to match the broader project brief. The model is trained only on measurements that exist in the selected dataset.	

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

7. Machine Learning Models and Training

Three classification models are trained so that the project is not dependent on a single algorithm. Logistic Regression gives a transparent linear baseline. Random Forest captures nonlinear interactions through an ensemble of decision trees. XGBoost provides a gradient-boosted tree approach and serves as another strong tabular-data reference.

Model	Training approach	Reason for inclusion
Logistic Regression	Median imputation → StandardScaler → LogisticRegression	Simple probability baseline
Random Forest	Median imputation → 300-tree RF, balanced class weighting	Nonlinear patterns + feature importance
XGBoost	Median imputation → boosted trees	Strong tabular classification comparison

The training split contains 80% of the records, while 20% is reserved for testing. A fixed random state of 42 is used. Stratification is enabled so that the class composition of the test data remains approximately consistent with the full dataset.

Random Forest is used as the main model for the health-monitoring demonstration because it combines probability prediction with a directly available global feature-importance representation. This is a practical engineering choice for an academic prototype: the model is not only able to classify an observation, it can also help explain which sensor variables matter to the learned decision structure.

Training stage	Action	Result
Feature selection	Drop Engine Condition from X	Sensor matrix
Target creation	Use Engine Condition as y	Binary labels
Split	80% / 20%, stratified	Train/test sets
Fit	Train each model	Three trained classifiers
Probability	predict_proba()	Failure/fault probability
Prediction	predict()	Class label
Evaluation	Compare metrics	Model comparison table

TRAINING DESIGN

The models are compared using the same train/test partition. This makes the reported metric differences easier to interpret because the algorithms are being evaluated against the same held-out observations.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

8. Model Evaluation and Comparative Results

The repository contains the completed comparison output for the three trained models. The metrics show that the current dataset supports a meaningful but imperfect classification task. XGBoost records the highest accuracy and ROC-AUC in the saved comparison, while Logistic Regression records the highest recall and F1 among the three models. These are descriptive results from the project's held-out test set, not a production benchmark.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	66.68%	69.91%	82.74%	75.79%	69.75%
Logistic Regression	66.11%	67.87%	87.82%	76.57%	69.18%
Random Forest	65.11%	68.57%	82.46%	74.88%	68.12%

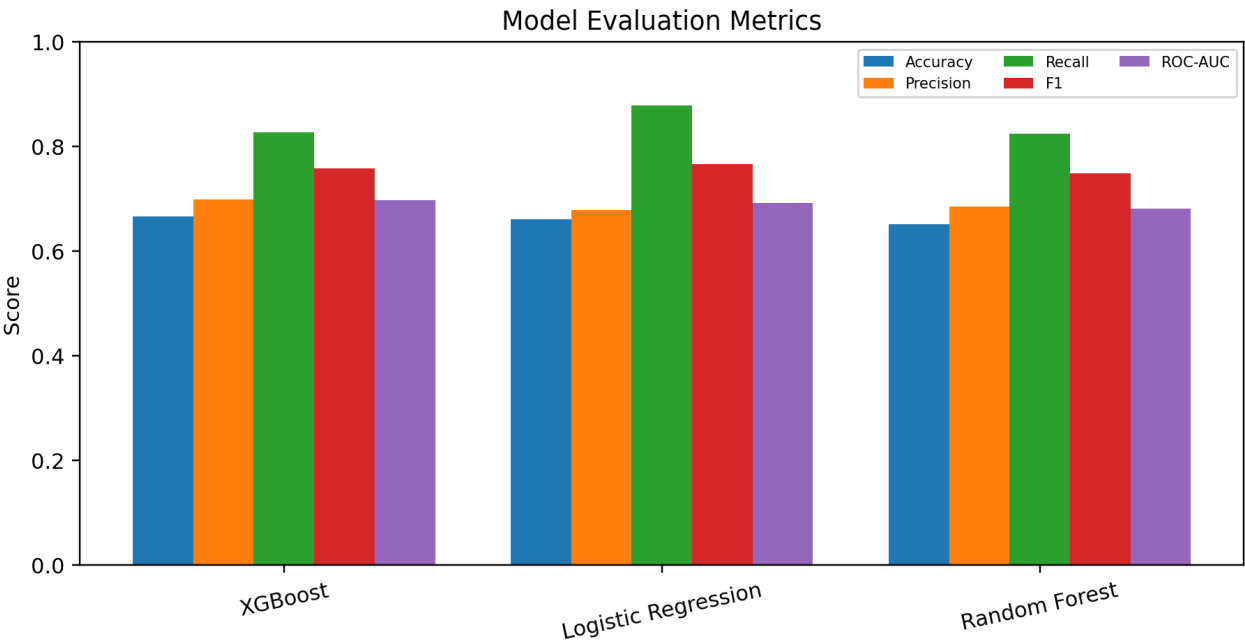


Figure 8.1. Model evaluation metrics reproduced from the repository's saved comparison results. [■cite■turn3view0■](#)

The comparison also demonstrates why several metrics should be retained. Accuracy alone would not describe the different trade-offs between false positives and false negatives. Recall, for example, is higher for Logistic Regression in the saved result, while XGBoost has the highest accuracy and ROC-AUC in the same test evaluation. [■cite■turn3view0■](#)

RESULT INTERPRETATION

The current results should be read as an evaluation of this dataset and implementation. They do not establish that one model will perform similarly on another vehicle, another sensor configuration or real fleet telemetry.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

9. Feature Importance and Explainable AI

A predictive-maintenance model becomes more useful when the engineer can see which measurements are influencing its learned decisions. Random Forest provides a global feature-importance measure. The repository records Engine rpm as the largest importance value at approximately 0.228, followed by Fuel pressure at approximately 0.165 and lub oil temp at approximately 0.159. [cite](#) [turn3view1](#)

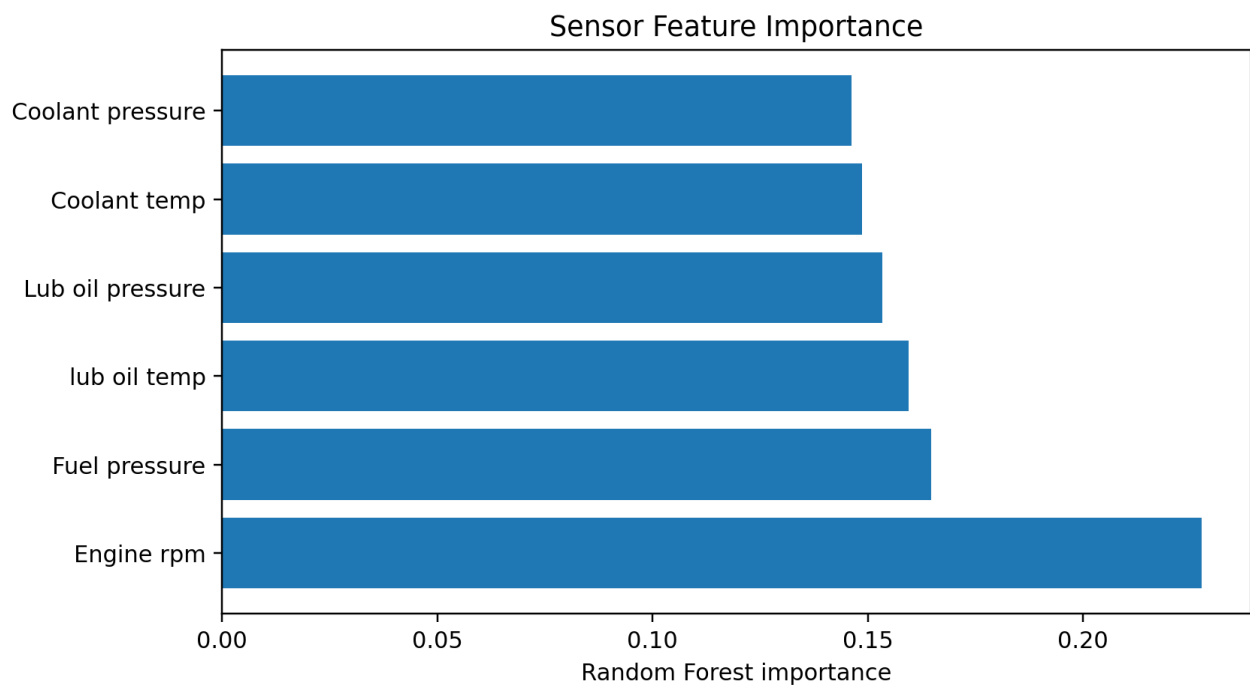


Figure 9.1. Random Forest feature importance from the repository output. [cite](#) [turn3view1](#)

Feature	Importance	Relative observation
Engine rpm	0.22764	Largest global importance in saved output
Fuel pressure	0.16472	Second-highest importance
lub oil temp	0.15945	Third-highest importance
Lub oil pressure	0.15330	Meaningful contribution
Coolant temp	0.14874	Meaningful contribution
Coolant pressure	0.14616	Meaningful contribution

The values are global model-level importance values. They do not mean that a sensor is mechanically 'bad' whenever it receives a high importance score. Instead, they describe how useful that variable was to the trained Random Forest's decision process across the evaluation problem.

XAI PRINCIPLE

Feature importance explains model structure; it should not be confused with a causal diagnosis of a mechanical fault. A production system would combine model explanations with engineering knowledge, sensor validation and component-level diagnostics.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

10. SHAP-Based Local Explanation

Global feature importance is useful when reviewing the whole model, but an engineer may also want to know what happened for one particular prediction. The notebook therefore includes an optional SHAP analysis. SHAP, or SHapley Additive exPlanations, represents how individual features contribute to a model output for a selected observation.

The saved SHAP output contains six feature contributions. Fuel pressure has a positive contribution of approximately 0.104, Engine rpm has approximately -0.071, Coolant pressure approximately 0.049, Lub oil pressure approximately 0.043, lub oil temp approximately -0.019 and Coolant temp approximately 0.007. ■cite■turn3view2■

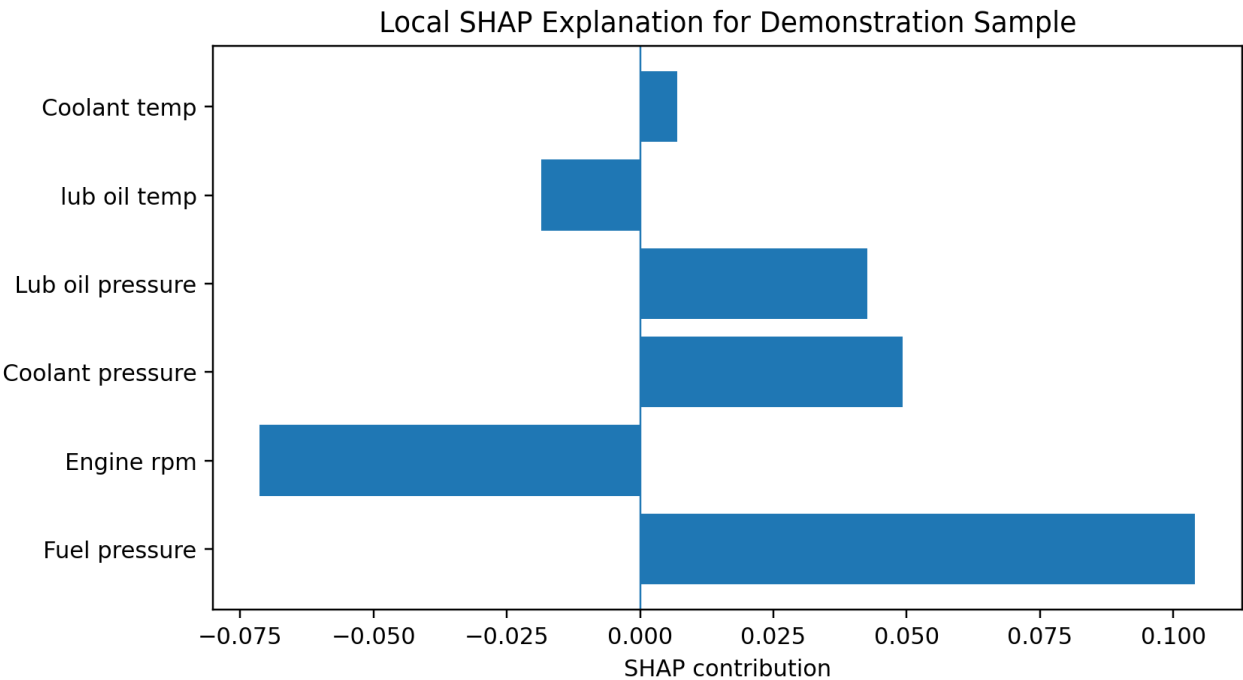


Figure 10.1. SHAP contribution values for the selected demonstration sample. ■cite■turn3view2■

Feature	SHAP contribution	Interpretation in the model explanation
Fuel pressure	+0.10413	Largest positive contribution for this sample
Engine rpm	-0.07134	Largest negative contribution for this sample
Coolant pressure	+0.04923	Positive contribution
Lub oil pressure	+0.04264	Positive contribution
lub oil temp	-0.01854	Negative contribution
Coolant temp	+0.00700	Small positive contribution

LOCAL EXPLANATION

The SHAP table is specific to the selected sample. It should not be generalized to every engine observation. That distinction between global importance and local explanation is essential when presenting XAI results.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

11. Failure Probability and Vehicle Health Score

The project does not stop at a class label. The trained classifier provides class probabilities, and the probability associated with the maintenance/fault class is retained as the project's demonstration failure probability. The health layer then converts that value into a percentage that is easier for a user to interpret.

The implemented relationship is:

Quantity	Formula
Failure probability	Model probability for the maintenance/fault class
Vehicle health score	$100 \times (1 - \text{failure probability})$

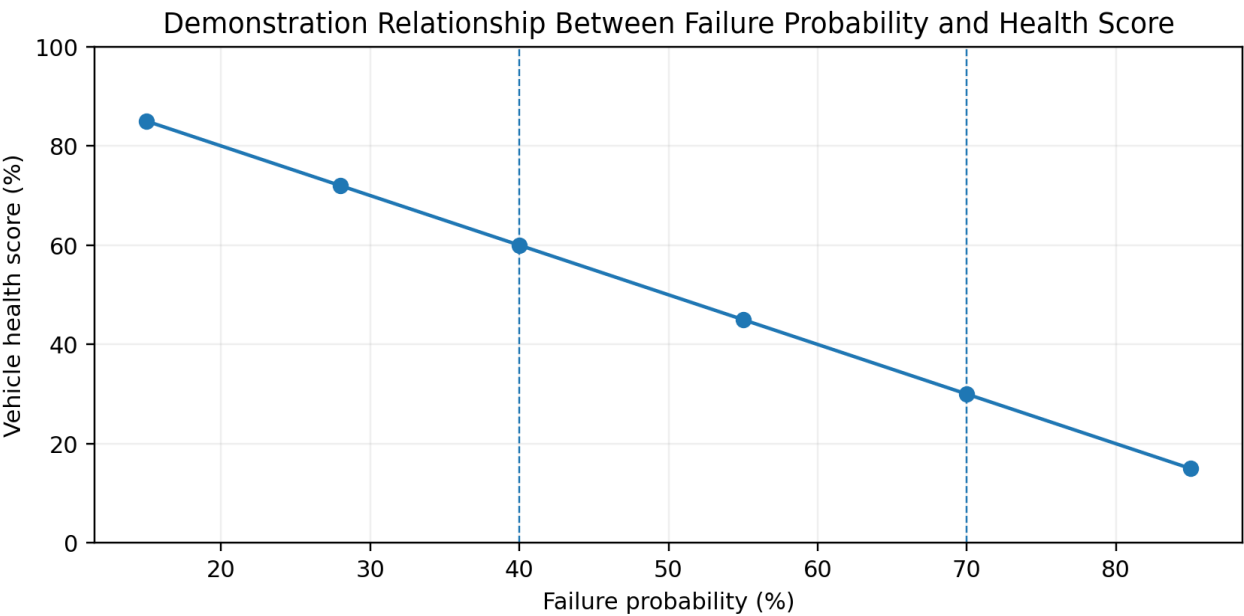


Figure 11.1. Demonstration relationship between failure probability and the derived health score.

For a failure probability of 0.28, the health-score calculation gives 72%. The project documentation uses this example to show how the model output can be translated into a human-readable monitoring value. ■cite■turn1view0■

Failure probability	Health score	Demonstration interpretation
15%	85%	Low model-estimated fault probability
28%	72%	Example health value
40%	60%	Boundary of WARNING range
55%	45%	WARNING range
70%	30%	Boundary of CRITICAL range
85%	15%	High demonstration risk

IMPORTANT QUALIFICATION

The health score is a derived project metric, not a certified vehicle-health measurement. The underlying probability should be calibrated and validated with representative fleet data before any production decision is based on it.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

12. Maintenance Status and Alert Generation

The monitoring layer converts the continuous probability into three simple states. A probability below 40% is treated as HEALTHY, 40% to below 70% as WARNING, and 70% or above as CRITICAL. These thresholds make the demonstration easy to read and provide a direct connection between the numerical model output and a maintenance message.

Status	Failure probability	Health score range	Maintenance message
HEALTHY	< 40%	> 60%	NO IMMEDIATE ACTION
WARNING	40% to < 70%	> 30% to ≤ 60%	INSPECTION REQUIRED
CRITICAL	≥ 70%	≤ 30%	INSPECTION REQUIRED

Input state	Processing	Visible output
Sensor row	Model prediction	Failure probability
Failure probability	$100 \times (1 - \text{probability})$	Vehicle health score
Probability + threshold	Status rule	HEALTHY / WARNING / CRITICAL
Status	Alert mapping	Maintenance message
Sensor row + model	Feature importance / SHAP	Contributing factors

The alert-generation function is deliberately simple. For a classroom or prototype demonstration, a readable state is preferable to a complicated decision engine that would suggest a level of automotive certainty the available data cannot support.

The repository's README describes the same mapping and notes that the thresholds are demonstration thresholds that should be calibrated using real fleet data and engineering requirements before use in an actual vehicle. [cite turn1view0](#)

OPERATOR-FACING OUTPUT

Vehicle Health → Failure Probability → Status → Maintenance Alert. The purpose is to reduce the amount of model-specific information an operator must interpret manually.

A future dashboard could extend this section with historical health trends, alert acknowledgement, maintenance history and fleet-level monitoring without changing the underlying classification interface.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

13. Testing and Observed Results

Testing focuses on deterministic software behaviour: whether the dataset loads, models train, predictions are generated, the health score is calculated and the expected output files are created. The repository already contains the generated comparison, feature-importance, SHAP and vehicle-health files, which provides a traceable record of the completed workflow.

■cite■turn0view0■

TC ID	Action	Expected result	Observed / documented result
TC-01	Load dataset	DataFrame created	Implemented
TC-02	Inspect columns	Six sensor features + target visible	Implemented
TC-03	Train Logistic Regression	Classifier fitted	Implemented
TC-04	Train Random Forest	Classifier fitted	Implemented
TC-05	Train XGBoost	Classifier fitted	Implemented
TC-06	Generate probabilities	Class probabilities available	Implemented
TC-07	Calculate health score	0–100 score produced	Implemented
TC-08	Generate status	HEALTHY/WARNING/CRITICAL returned	Implemented
TC-09	Generate XAI output	Importance + SHAP available	Implemented
TC-10	Export artifacts	CSV/model/summary files created	Implemented

The saved model-comparison file contains the three evaluation rows used in this report. The saved feature-importance file contains six sensor variables, while the SHAP file contains six local contributions. ■cite■turn3view0■turn3view1■turn3view2■

A useful testing habit in this project is to treat the output artifacts themselves as evidence. If a training run finishes but the expected CSV or model file is absent, the workflow is not complete even if the notebook appears to have executed successfully.

PASS CRITERION

A test is considered functionally demonstrated when the corresponding pipeline stage executes, produces the expected type of output and leaves a result that can be inspected or reused.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

14. Implementation Process and Design Decisions

The implementation was developed in stages so that each part could be inspected before the next layer was added. The final repository reflects this incremental approach: the notebook contains the full workflow, while the repository stores the resulting model-analysis artifacts separately.

Stage	Work completed	Reason
1	Select public engine-sensor dataset	Provide reproducible structured input
2	Build Colab loading and inspection cells	Confirm data shape and columns
3	Add preprocessing pipeline	Handle missing values consistently
4	Train Logistic Regression	Create baseline
5	Train Random Forest	Add nonlinear model and importance
6	Train XGBoost	Add boosted-tree comparison
7	Add evaluation metrics	Measure held-out performance
8	Add health-score logic	Translate probability into user output
9	Add XAI	Explain model behaviour
10	Export artifacts	Make results reusable and shareable

One important design decision was to avoid manufacturing missing automotive signals simply to make the project appear broader. The project brief mentions vibration, battery voltage, vehicle speed, fuel consumption and operating hours, but the selected dataset does not contain them. Keeping the implementation limited to real dataset fields makes the experiment easier to reproduce and defend.

Another design decision was to keep the health-score calculation separate from the classifier. The model is responsible for learning a classification boundary and producing a probability. The monitoring layer is responsible for deciding how that probability should be displayed. This separation allows the threshold policy to be changed later without retraining the classifier.

DESIGN DECISION

Simulation and model demonstration are kept separate from production automotive integration. The current project proves the machine-learning workflow; it does not claim to have implemented a live ECU, CAN-bus gateway or certified diagnostic controller.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

15. Project Outputs and Reproducibility

The repository contains the main notebook together with the files produced by the completed workflow. The GitHub repository currently lists the dataset link, five Mod3 images, the Colab notebook, README, SHAP explanation, feature importance, model comparison results, project summary and vehicle-health predictions. [■cite■turn0view0■](#)

Artifact	Purpose
Predictive_Maintenance_Vehicle_Health_Monitoring_Colab.ipynb	Complete executable Google Colab workflow
DATASET_LINK.txt	Dataset source and direct CSV reference
model_comparison_results.csv	Accuracy, precision, recall, F1 and ROC-AUC
feature_importance.csv	Random Forest global feature importance
SHAP_explanation.csv	Local SHAP contributions
vehicle_health_predictions.csv	Predicted conditions and monitoring outputs
project_summary.txt	Short project-level summary
Predictive_Maintenance_Project_All_Outputs.zip	Packaged project outputs
README.md	Detailed repository documentation

The notebook is designed to be executed in Google Colab. Because the dataset is loaded from a direct public URL, the user does not need to manually upload the raw CSV before running the data-loading cell. The notebook also uses a fixed random state, making the train/test partition reproducible under the same environment.

The final output ZIP workflow is particularly useful for submission. It can collect generated graphs, model-comparison tables, prediction CSVs, XAI outputs, the serialized model and a project summary into a single downloadable archive.

Reproducibility item	Current implementation
Random seed	42
Test size	20%
Split type	Stratified
Dataset source	Direct public CSV URL
Environment	Google Colab / Python
Saved model format	Joblib .pkl
Tabular result format	CSV

REPOSITORY EVIDENCE

The public GitHub repository provides the executable notebook and saved result artifacts, allowing the report's numerical model results and XAI tables to be traced back to project files. [■cite■turn0view0■](#)

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

16. Validation, Limitations and Engineering Assumptions

Validation in the current project is primarily software and dataset based. The notebook successfully defines a complete pipeline from data acquisition through model comparison and output generation. However, the available evidence does not establish that the model is ready for direct use in a real vehicle. That distinction is central to responsible automotive machine learning.

Area	Validated in project	Limitation / assumption
Data loading	Public engine dataset loaded	Dataset is not a live vehicle stream
Preprocessing	Missing-value handling implemented	Sensor quality and calibration are not field-tested
Classification	Three ML models trained	Generalisation to other fleets is unverified
Probability	Class probabilities produced	Calibration for real failure rates not demonstrated
Health score	Derived 0–100 score produced	Not a certified automotive health index
Alerts	Threshold logic implemented	Thresholds are demonstration values
XAI	Importance and SHAP generated	Explanations are model-based, not mechanical diagnosis
Vehicle integration	Conceptual architecture only	No CAN/OBD-II live integration in current repo

IMPORTANT ASSUMPTION

The project is an academic software demonstration. It should not be interpreted as a production-ready automotive diagnostic system, safety controller or validated mechanical-failure predictor.

The model comparison itself also shows why validation matters. Accuracy values in the saved results are in the mid-60% range, so there is substantial room for improvement before a system could be expected to support high-consequence decisions.

A stronger validation programme would use temporally separated fleet data, vehicle-level holdout sets, sensor fault injection, probability calibration, repeated cross-validation and independent testing on vehicles not represented during training. Engineering validation would then be needed to determine which alerts are genuinely useful to maintenance personnel.

The current project should therefore be understood as a foundation: it proves the software workflow and creates a clear place for better data, better models and real automotive integration to be introduced.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

17. Future Scope and Advanced Development Roadmap

The current prototype is intentionally small, but its architecture can be extended without discarding the existing data-to-health concept. The most important next step is richer and time-aware telemetry. A single row of sensor measurements is useful for classification, but maintenance decisions often depend on trends: a temperature that is steadily increasing can carry different information from a temperature that is temporarily high.

Future stage	Extension	Expected engineering value
1	Add vibration and battery voltage	Broader physical-condition coverage
2	Add vehicle speed, consumption and operating hours	Context-aware vehicle behaviour
3	Real-time CAN/OBD-II ingestion	Move from offline CSV to live telemetry
4	LSTM / GRU	Learn temporal sensor patterns
5	Transformer models	Model longer and richer sensor sequences
6	Probability calibration	Make risk values more interpretable
7	RUL estimation	Estimate remaining useful life
8	Dashboard / Android application	Make monitoring accessible to operators
9	Edge inference	Reduce latency and connectivity dependence
10	Fleet-level analytics	Compare vehicles and maintenance patterns

A future time-series architecture could be represented as:

LIVE SENSORS	→	WINDOWED TELEMETRY	→	SEQUENCE MODEL	→	HEALTH / RUL
RPM / pressure / temperature / vibration	→	Recent sensor history	→	LSTM / GRU / Transformer	→	Risk + health + maintenance

The repository's current architecture already separates model prediction from the monitoring layer. That makes it possible to replace the classical classifier with a temporal model later while preserving the idea of a failure probability, health score and maintenance message.

EXTENSIBILITY

The cleanest development path is incremental: improve the data first, validate the target definition, calibrate probabilities, then add temporal modelling and live connectivity. Each stage should be validated independently rather than adding complexity all at once.

ENGINEERING NOTE

The implementation is intentionally traceable: each numerical output can be connected to the underlying sensor data, model stage and saved project artifact. This makes the prototype easier to reproduce, explain and extend without presenting future automotive functionality as already implemented.

A useful way to read this stage is to ask three questions: what information entered the stage, what transformation was applied, and what evidence was produced. The project follows this pattern throughout the notebook, which helps separate measured results from assumptions and future scope.

18. Conclusion

The AI/ML-Based Predictive Maintenance and Vehicle Health Monitoring System demonstrates a complete and traceable machine-learning workflow for engine-condition analysis. The project begins with a public engine-sensor dataset and ends with a user-oriented representation of model output. In between, it covers data inspection, preprocessing, model training, evaluation, explainability and artifact generation.

The implementation compares Logistic Regression, Random Forest and XGBoost. The saved repository results show measurable differences across accuracy, precision, recall, F1 and ROC-AUC, making the model-comparison stage a meaningful part of the project rather than an afterthought. XGBoost records 66.68% accuracy and 69.75% ROC-AUC in the saved test results, while Logistic Regression records 87.82% recall and 76.57% F1. Random Forest is then used for the vehicle-health demonstration and explainability workflow.

The system translates the maintenance/fault probability into a health score using a simple inverse relationship and then maps the result to HEALTHY, WARNING or CRITICAL states. The project also goes one step further by showing which features matter globally through Random Forest importance and how individual features contributed to a selected prediction through SHAP.

Final project status	Status
Dataset acquisition	Implemented
Data preprocessing	Implemented
Logistic Regression	Implemented and evaluated
Random Forest	Implemented, evaluated and used for monitoring demonstration
XGBoost	Implemented and evaluated
Failure probability	Implemented
Vehicle health score	Implemented
Maintenance status / alert	Implemented
Feature importance	Implemented
SHAP explanation	Implemented
Repository artifacts	Published on GitHub

USER INPUT	→	SENSOR FEATURES	→	ML MODEL	→	HEALTH / ALERT
Engine observation	→	RPM + pressures + temperatures	→	LR / RF / XGBoost	→	Probability + score + status

Figure 18.1. Final conceptual predictive-maintenance pipeline represented by the project.

FINAL ENGINEERING STATEMENT

The project meets its educational objective by demonstrating an end-to-end predictive-maintenance workflow with real dataset inputs, multiple machine-learning models, measurable evaluation results, health-score generation and Explainable AI. The next engineering step is not simply a larger model; it is stronger vehicle data, calibration and validation followed by live automotive integration.

Project repository: <https://github.com/SAGNIK890/Predictive-Maintenance-and-Vehicle-Health-Monitoring-System>
Dataset: <https://huggingface.co/datasets/mukherjee78/predictive-maintenance-engine-data>